

```

def is_safe(maze, row, col, visited):

    rows = len(maze)
    cols = len(maze[0])

    return (0 <= row < rows and
            0 <= col < cols and
            maze[row][col] == 0 and
            (row, col) not in visited)

def dfs_solver(maze, current, end, visited, path):

    # Add current node to path and mark as visited
    path.append(current)
    visited.add(current)

    # Base Case: Reached the destination
    if current == end:
        return True

    row, col = current

    # Define possible moves: Down, Right, Up, Left
    moves = [(1, 0), (0, 1), (-1, 0), (0, -1)]

    for dr, dc in moves:
        new_row, new_col = row + dr, col + dc

        if is_safe(maze, new_row, new_col, visited):
            if dfs_solver(maze, (new_row, new_col), end, visited, path):
                return True

    # Backtracking: If no move leads to the end, remove current from path
    path.pop()
    return False

def solve(maze, start_point, end_point, case_name):

    print(f"--- Running {case_name} ---")

    path = []
    visited = set()

    # Check if start or end points are actually walls before starting
    if maze[start_point[0]][start_point[1]] == 1 or maze[end_point[0]][end_point[1]] == 1:
        print("No path found (Start or End is blocked)\n")
        return

    # Start the DFS execution
    if dfs_solver(maze, start_point, end_point, visited, path):
        print("Path found:")
        print(path)
    else:
        print("No path found")
        print("\n")

if __name__ == "__main__":
    #EXAMPLE 1
    maze_1 = [
        [0, 1, 0, 0, 0],
        [0, 1, 0, 1, 0],
        [0, 0, 0, 1, 0],
        [1, 1, 1, 0, 0],
        [0, 0, 0, 0, 0]
    ]
    start_1 = (0, 0)
    end_1 = (4, 4)

    solve(maze_1, start_1, end_1, "Example 1")

    #EXAMPLE 2
    maze_2 = [
        [0, 1, 1, 1, 1],
        [1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1],
        [1, 1, 1, 1, 0]
    ]
    start_2 = (0, 0)
    end_2 = (4, 4)

```

```
solve(maze_2, start_2, end_2, "Example 2")
```

```
--- Running Example 1 ---
```

```
Path found:
```

```
[(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (1, 2), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]
```

```
--- Running Example 2 ---
```

```
No path found
```